

```

GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    int i;
    pid_t pid;

    printf("Parent PID: %d\n", getpid());

    for (i = 1; i <= 3; i++)
    {
        pid = fork();

        if (pid < 0)
        {
            perror("fork failed");
            return 1;
        }

        if (pid == 0)
        {
            printf("Child %d started. PID = %d\n", i, getpid());

            sleep(i * 2);

            printf("Child %d completed. PID = %d\n", i, getpid());

            exit(0);
        }
    }

    for (i = 1; i <= 3; i++)
    {
        wait(NULL);
        printf("Parent collected a child.\n");
    }

    printf("All children completed.\n");

    return 0;
}

```

```

vootla_hemanth@DESKTOP-BT7MELO:~$ ./week4_wait
Parent PID: 877
Child 1 started. PID = 878
Child 2 started. PID = 879
Child 3 started. PID = 880
Child 1 completed. PID = 878
Parent collected a child.
Child 2 completed. PID = 879
Parent collected a child.
Child 3 completed. PID = 880
Parent collected a child.
All children completed.
vootla_hemanth@DESKTOP-BT7MELO:~$ |

```

```

GNU nano 8.7.1                                     week4_waitpid.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    int i;
    pid_t pid;
    pid_t children[3];

    printf("Parent PID: %d\n", getpid());
    for (i = 0; i < 3; i++)
    {
        pid = fork();
        if (pid < 0)
        {
            perror("fork failed");
            return 1;
        }
        if (pid == 0)
        {
            printf("Child %d started. PID = %d\n", i + 1, getpid());
            sleep((i + 1) * 2);
            printf("Child %d completed. PID = %d\n", i + 1, getpid());
            exit(10 + i);
        }
        children[i] = pid;
    }
    for (i = 0; i < 3; i++)
    {
        int status;
        waitpid(children[i], &status, 0);
        if (WIFEXITED(status))
        {
            printf("Child PID %d completed with status %d\n",
                    children[i],
                    WEXITSTATUS(status));
        }
    }
    printf("Parent process completed.\n");
    return 0;
}

```

```

vootla_hemanth@DESKTOP-BT7MELO:~$ ./week4_waitpid
Parent PID: 923
Child 1 started. PID = 924
Child 2 started. PID = 925
Child 3 started. PID = 926
Child 1 completed. PID = 924
Child PID 924 completed with status 10
Child 2 completed. PID = 925
Child PID 925 completed with status 11
Child 3 completed. PID = 926
Child PID 926 completed with status 12
Parent process completed.
vootla_hemanth@DESKTOP-BT7MELO:~$ |

```

```

GNU nano 8.7.1 week4_zombie
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process\n");
        printf("Child PID: %d\n", getpid());
        printf("Child is terminating...\n");

        exit(0);
    }
    else
    {
        printf("Parent process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent is sleeping...\n");

        sleep(30);

        printf("Parent completed.\n");
    }

    return 0;
}

```

```

~$ bash: ./week4_zombie: No such file or directory
vootla_hemanth@DESKTOP-BT7MELO:~$ ./week4_zombie
Parent process
Parent PID: 976
Child PID: 977
Parent is sleeping...
Child process
Child PID: 977
Child is terminating...
Parent completed.
vootla_hemanth@DESKTOP-BT7MELO:~$ |

```